Enunciado Tarea tema 5: Gestión de Torneo de Fútbol con PySide6 y SQLite

Título: Desarrollo de una aplicación de escritorio para la gestión de un torneo de fútbol en el Instituto.

Descripción:

El objetivo de esta práctica es diseñar e implementar una aplicación de escritorio para gestionar un torneo de fútbol de eliminatorias. La aplicación permitirá a los usuarios realizar tareas como registrar equipos, asignar árbitros, programar partidos, actualizar resultados y mostrar la clasificación. Utilizaremos **Python**, **PySide6**, **Qt Designer** y una base de datos SQLite para desarrollar la solución.

Se tendrá en cuenta que la inclusión de imágenes e iconos en las interfaces desarrolladas. Se

debe independizar lo máximo posible control de datos y vistas.

No se proporciona base de datos. Ésta deberá ser creada por el alumno y documentada: tablas, claves principales y relaciones entre tablas.

Se proporciona un documento adicional con algunas indicaciones para la creación de la base de datos, inclusión del qss y distribución de la aplicación.

El proyecto deberá entregarse en formato ejecutable (.exe) completamente funcional, de modo que el usuario final pueda ejecutar la aplicación directamente mediante doble clic, sin necesidad de realizar ninguna configuración previa ni posterior. **La calificación será de 0 para cualquier proyecto que no funcione de esta manera.**

Se deberá incluir un **archivo README** que contenga todas las consideraciones o las instrucciones necesarias antes de ejecutar el programa (en desarrollo y distribuido).

Requisitos de la práctica

1. Funcionalidades básicas

1. **Gestión de Equipos**:
    - Crear nuevos equipos indicando nombre, curso y color de camiseta y otra información que se estime necesaria, como por ejemplo emblema/logo/escudo.
    - Listar todos los equipos registrados y si se pulsa sobre un equipo listará los jugadores de ese equipo.
    - Editar o eliminar equipos existentes.
2. **Gestión de Participantes**:
    - Registrar participante con datos como nombre, fecha nacimiento, curso, jugador y/o árbitro (el participante puede ser jugador o árbitro o ambos), posición (delantero, defensa, centrocampista y portero…), tarjetas amarillas, tarjetas rojas, goles .
    - Decidir si se asigna un jugador a un equipo o desde el equipo se seleccionan los jugadores del equipo o ambas situaciones.
    - Decidir si el árbitro se asigna a un partido o al partido se asigna un árbitro o ambas situaciones.
3. **Gestión del Calendario**:
    - Programar partidos indicando:
        - Equipos participantes.
        - Fecha y hora.

        - Árbitro asignado.
        - Eliminatorio (previa, dieciseisavos, octavos, cuartos, semi y final). Al menos comenzar en octavos.
    - Listar los partidos programados en orden de fechas y eliminatoria. No es necesario una lista, puede ser un árbol con las eliminatorias...
    - Editar o eliminar partidos.
4. **Actualización de Resultados**:
    - Permitir registrar los goles de cada equipo para actualizar automáticamente la clasificación.
    - Registrar goles y tarjetas a los jugadores.
    - Listar los partidos jugados con resultado y eliminatoria. Puede ser la misma que en Gestión de calendario...
    - Listar, editar y eliminar participantes.
    - Listar o clasificar por tarjetas o goles.
5. **Clasificación o Gestión de eliminatorias:**
    - Generar automáticamente las rondas de cuartos de final, semifinales y final según los resultados.
    - Mostrar el cuadro de emparejamiento por eliminatorias.
    - Se puede unificar con los resultados de Gestión de calendario o Actualización de resultados.
6. **Créditos**
    - Pantalla con información sobre el programa como autor, versión, fecha de actualización…
7. **Ayuda**
    - Ventana con ayuda mínima para utilizar la aplicación

---

2. Requisitos técnicos

1. **Interfaz gráfica**:
    - Utilizar **Qt Designer** para diseñar las ventanas.
    - Cargar los diseños en Python con PySide6.
2. **Base de datos**:
    - Implementar SQLite para almacenar la información de equipos, árbitros, partidos y resultados.
    - Gestionar las relaciones entre tablas (por ejemplo, partidos y equipos).
    - La base de datos debe ser creada y documentada: tablas, claves principales, relaciones entre tablas, restricciones...
3. **Código limpio**:
    - Separar la lógica de la interfaz (control) y la gestión de la base de datos en módulos/clases diferentes (modelo) de las interfaces (vistas). Hacer lo más independiente posible control de datos y vistas.
    - Documentar el código y seguir las buenas prácticas de programación (docstring).
4. **Estilo visual y empaquetado**:
    - Personalizar el estilo de la aplicación usando hojas de estilo de Qt (QSS).
    - Añadir iconos en los botones e imágenes en las ventanas

- Tooltips
- Empaqueta la aplicación usando **PyInstaller** para Windows **y deb** para GNU/Linux
- Utiliza el módulo de acceso a la base de datos y cálculo de rutas absolutas como una librería local instalable. No lo incluyas como un módulo en el proyecto. Documenta como se debe instalar esta librería.

---

3. Entregables

1. Código fuente organizado en carpetas.
    - `Views`
        - ui/: Archivos `.ui` diseñados en Qt Designer.
        - Archivos .py con las vistas en código Python
    - `Controllers`: Módulos Python con la lógica de la aplicación.
    - `Models`
        - Archivo SQLite utilizado por la aplicación.
        - Clases necesarias para manejar las tablas
        - Clase para manejar el acceso vía SQL a la base de datos.
    - `Resources.`
        - img. Imágenes utilizadas
        - iconos. Imágenes de iconos utilizados
        - qss. Archivo/s qss
2. Manual de usuario breve en formato PDF. Incluir información sobre base de datos y sobre librerías propias que se deben instalar.
3. Informe técnico explicando la estructura del proyecto y las decisiones tomadas durante el desarrollo.

Criterios de evaluación

Criterio	Puntuación
Diseño de la interfaz gráfica	30%
Funcionalidades implementadas	20%
Uso de base de datos y relaciones	10%
Calidad y claridad del código	10%
Estilo visual y usabilidad	15%
Documentación (manual e informe)	5%
Empaquetado	10%

Opcional (valoración a la alza)

1. **Exportación de datos**: Permitir exportar la clasificación o los resultados a un archivo CSV.
2. **Notificaciones**: Mostrar alertas al usuario (por ejemplo, cuando falta asignar árbitros o completar resultados).

Sugerencias para los alumnos:

- Identifica las relaciones de la base de datos antes de comenzar el desarrollo.
- Divide las tareas en etapas y prioriza las funcionalidades básicas antes de implementar extensiones.
- Asegúrate de probar la aplicación con diferentes datos y que se guarden correctamente.
- Antes de entregar el proyecto, asegúrate de verificar que el proceso descrito en el README permite ejecutar la aplicación correctamente en cualquier máquina siguiendo únicamente los pasos indicados.